

Week 8: Sift: A Domain-Specific Language for Nightly CSV to JSON Ingestion

Aaron Foster

CS 660: Advanced Programming Language Theory

Southern New Hampshire University

Prof. Shadha Tabatabai

August 28th, 2026

Why a General-Purpose Language Is Not Enough

While standard Python using the built-in csv and json modules can easily convert delimited files into structured data within fifty lines of code, the true justification for a domain-specific language emerges as ingestion scales. Rather than acting as a singular application, nightly data processing functions as a suite of closely related programs—one for each data partner—where differences lie solely in field names, data types, and validation logic. As Mernik et al. (2005) point out, recurring software tasks characterized by notational rather than algorithmic variation represent the precise domain where developing a DSL yields a net benefit.

Three failures follow when that family lives in a general-purpose language. The mapping rules become unreadable to the people who own them: the analyst who knows whether Qty may be blank cannot say which columns a feed requires without reading imperative code interleaved with file handles and loop control. The defaults are wrong for the domain, since `int("")` raises, so every script grows ad hoc rescue logic and the safe response — quarantine the row, continue, abort only past a threshold — is rebuilt differently each time. And nothing is checkable before execution, so no artifact lets a reviewer or a continuous integration job confirm that the output will match the agreed schema. RFC 4180 compounds this, documenting CSV practice rather than binding a standard (Shafranovich, 2005).

Sift is deliberately less powerful than Python: no loops, no functions, no arbitrary expressions. That restriction is the product, not a defect; limited expressiveness is what distinguishes a domain-specific language from a library (Fowler, 2010). Because a Sift program can only describe a row-wise mapping, it is also a machine-readable schema contract — readable

by a domain expert, checkable before the run, and executed by one hardened runtime instead of forty independently buggy scripts.

The Language Surface

Sift exposes five constructs. Figure 1 is a complete program for a nightly ordered feed; each construct is examined below by its runtime behavior and its fit to the domain.

Figure 1. A complete Sift program for a nightly orders feed.

```
# orders_nightly.sift

SOURCE "s3://feeds/orders/{{run_date}}.csv"
  DELIMITER ","  HEADER required  ENCODING utf-8

FIELD order_id      FROM "OrderID"    AS string      REQUIRED
FIELD placed_at     FROM "OrderDate"  AS date("MM/DD/YYYY") REQUIRED
FIELD quantity      FROM "Qty"        AS integer      DEFAULT 0
FIELD unit_price     FROM "UnitPrice"  AS decimal(10,2)   REQUIRED
FIELD customer_id    FROM "CustID"     AS string        REQUIRED
FIELD customer_zip   FROM "Zip"        AS string        DEFAULT
"00000"

REQUIRE quantity    >= 0              ON FAIL reject_row
REQUIRE unit_price  > 0.00            ON FAIL reject_row
REQUIRE placed_at   <= run_date ON FAIL halt_run

NEST customer FROM customer_id, customer_zip
  AS { "id": customer_id, "zip": customer_zip }

EMIT TO "s3://lake/orders/{{run_date}}.ndjson"  FORMAT ndjson
REJECTS TO "s3://quarantine/orders/{{run_date}}.csv"
ON REJECT RATE > 2% halt_run
```

SOURCE: Binding the Input Contract

Runtime behavior. SOURCE runs once, before any row is read. The runtime interpolates the run date, opens the path as a stream rather than loading it into memory, decodes with the declared encoding, reads the header into a name-to-position index, and verifies that every column named by a FIELD is present. A missing column aborts the run before a record is written, naming the column and the header actually supplied. **Domain alignment.** Unattended runs make silent partial success the expensive failure, not a crash. Checking the header contract

first turns the commonest upstream change, a renamed column, into an immediate and legible one, and declaring the delimiter rather than sniffing it keeps the run deterministic.

FIELD: Typed, Explicit Column Mapping

Runtime behavior. Each FIELD compiles to one coercion bound to a fixed column position. Per row it reads the cell, applies the conversion, and writes the result under the field name. A blank cell takes DEFAULT; a blank REQUIRED cell, or text that will not coerce, raises a typed error carrying the field name, the offending text, the row number, and the statement's source line, which the row loop routes by the reject policy rather than terminating. A column named by no FIELD never reaches the output; there is no wildcard passthrough. **Domain alignment.** One line per output key makes the program its own schema documentation, and the absence of passthrough guarantees that a new upstream column cannot silently widen the JSON. Explicit types also move ambiguous cases, such as MM/DD/YYYY against DD/MM/YYYY, out of a library's heuristics and into a declaration someone agreed to.

REQUIRE: Validation With a Declared Failure Policy

Runtime behavior. REQUIRE compiles to a predicate over already-coered values, so it runs after every FIELD for that row and works on types rather than text. Predicates evaluate in declaration order and short-circuit on the first failure. Then reject_row writes the raw line to quarantine with the reason and the failed predicate's source line and advances, while halt_run aborts the job with the destination untouched. **Domain alignment.** Not all bad data is equally bad, and the domain expert rather than the runtime is qualified to say so. A negative quantity is one bad record, and quarantining it lets four million good ones load on schedule; a future order date means a broken clock or the wrong file, so loading any row is worse than none. The per-rule clause forces that judgment at authoring time.

NEST: Shaping Flat Rows Into Structured JSON

Runtime behavior. NEST runs after FIELD and REQUIRE, on the in-memory record only. It removes the listed fields from the top level and inserts a nested object under the given key in the literal shape of the AS clause, which admits object and array literals and field references but no computation. Because it consumes its sources, a field cannot appear both nested and at top level. **Domain alignment.** The gap between the formats is structural: CSV is flat, JSON hierarchical, and consumers expect a customer object rather than two prefixed scalars. Restricting NEST to a literal shape keeps the transformation legible, since the AS clause looks like the JSON it produces, and keeps the output shape derivable without running the program.

EMIT: Output, Quarantine, and Run-Level Policy

Runtime behavior. EMIT runs once per row to serialize and once at the end to finalize. Per row it appends one newline-delimited JSON object to a buffered writer aimed at a temporary location. At the end it applies the reject-rate policy: over threshold it discards that file, leaves the destination untouched, and exits non-zero; otherwise it moves the file atomically into place and writes a manifest of rows read, emitted, and rejected. **Domain alignment.** Partial output is worse than none, because a downstream job cannot tell a truncated file from a complete one, so the atomic move makes the destination all-or-nothing. A low reject rate is routine while a high one means the feed changed shape, and the threshold states that boundary as a number the business owner chooses.

Verbosity Against Flexibility

Sift is unapologetically verbose: a sixty-column feed needs sixty FIELD lines. The obvious remedy is a wildcard such as FIELD * AS string, tempting because most columns are strings that pass through untouched. It is rejected, because the verbosity is where the guarantees

live. A wildcard reintroduces the exact failure the language exists to prevent — a partner adds a column, it flows silently into the JSON, and there is no change to the program, no diff to review, and no signal to a consumer. Enumeration also keeps review in reach of the analysts who should be doing it: a change from `REQUIRED` to `DEFAULT` is a one-line diff they can evaluate.

The tradeoff runs both ways, and rigidity is how DSLs die. When the language cannot express the twentieth feed's requirement, users write a Python script instead, and the fleet fragments again. Reusable `INCLUDE` files answer part of this by factoring out shared settings and field groups, attacking repetition rather than explicitness. The escape hatch, though, belongs at the language boundary, not inside it: Sift will not grow an embedded expression language or a call into host code, because either makes the output shape undecidable and collapses every static guarantee at once. Voelter (2013) treats this as the governing question in DSL design, since expressiveness added late under pressure from one hard case usually costs more than the case was worth.

Interpreted or Compiled Execution

The workload sets the terms. A nightly run processes a large, predictable volume in one batch, so startup latency is irrelevant and throughput matters only insofar as the job finishes inside its window. But per-row cost is paid once per field per row, so a twenty-million-row feed with thirty fields performs six hundred million field operations, and constant factors invisible in an interactive tool become decisive.

A tree-walking interpreter is simplest to build and easiest to diagnose, but it re-dispatches on node type for each field of each row, repeating work whose answer was fixed at parse time: resolving column positions, selecting the coercion, re-reading the date format. Compiling to a standalone executable removes that overhead but fits the domain poorly, putting a build artifact

between the author and the run, so what executes at two in the morning is no longer what the analyst reviewed and failures surface as traces through generated code rather than a reference to line 9 of the program.

Staged interpretation takes the throughput without the artifact. The program is parsed and analyzed once, then lowered into a flat list of prepared operations — each a closure that has already captured its column index, coercion function, and parsed date format — before the first row is read. The row loop then iterates that list with no interpretive dispatch. Per-program work happens once and only per-row work repeats — the separation of static from dynamic work that Aho et al. (2006) describe as the purpose of the analysis phases, obtained without a binary.

Recommended Implementation Approach

The recommendation is a staged interpreter built on a hand-written lexer and recursive-descent parser, shipped as a host-language library rather than a transpiler. Recursive descent suits this grammar because every statement opens with a distinct keyword, so one token of lookahead selects the production and the grammar is LL(1) by construction. A parser generator would add a build step and charge for it where it matters most: generated parsers report failures in terms of the grammar, while a hand-written one can say that a FIELD statement is missing its FROM clause and show the line.

Execution proceeds in four stages: the lexer produces tokens carrying source spans, the parser builds a syntax tree, a semantic pass validates the whole program before anything runs — unique field names, resolvable references, DEFAULT literals matching declared types — and a lowering pass compiles the validated tree into the prepared operation list. Transpiling to Python is rejected on the same grounds, since generated Python inherits Python's error semantics and reintroduces the defaults the DSL exists to replace. Because analysis is already a separate phase,

a sift check command can run the front end against a sample header in continuous integration, reporting every structural error without touching production data.

Implementation Challenge: Diagnostics Across a Staged Pipeline

The hardest engineering problem follows from the central optimization. Staging destroys the link between a runtime failure and the source text: once a FIELD statement is lowered into an anonymous closure, a coercion failure on row 4,812,003 occurs inside a function that no longer knows it came from line 9. The default outcome is a host-language stack trace through interpreter internals: unreadable to the analyst who owns the mapping and, since the job runs unattended overnight, the only information anyone has when the alert opens.

The mitigation treats source location as data that survives every stage, and message quality as tested behavior. Every token carries a source span from the lexer onward, and the parser attaches spans to syntax nodes instead of discarding them. The lowering pass emits each prepared operation as a record pairing the closure with its span and construct name, so location travels with the operation at no per-row cost. The row loop renders caught errors through one formatter with a fixed shape — file and line, failing construct, offending value, row and column — as in "orders_nightly.sift:9: field 'quantity' expected integer, got 'n/a' (row 4812003, column 'Qty')". A golden-file corpus of broken programs then asserts those messages verbatim, so a refactor that degrades a diagnostic fails the build exactly as one that breaks a coercion would.

Conclusion

By introducing five core constructs alongside a staged interpreter, Sift intentionally refrains from expanding into a general-purpose programming language. Its core strength stems not from surpassing Python's functional capabilities, but from systematically eliminating common failure modes: undocumented transformations, unannounced schema drift, incomplete

data outputs, and fragmented error-handling strategies across growing script repositories.

Because rigidity poses a greater operational threat than runtime overhead, initial deployment should focus on a single prototype feed to evaluate whether domain analysts can independently read and modify the code.

References

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson Addison-Wesley.

Fowler, M. (2010). *Domain-specific languages*. Addison-Wesley.

Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), 316–344.

<https://doi.org/10.1145/1118890.1118892>

Shafranovich, Y. (2005). Common format and MIME type for comma-separated values (CSV) files (RFC 4180). Internet Engineering Task Force. <https://doi.org/10.17487/RFC4180>

Voelter, M. (2013). *DSL engineering: Designing, implementing and using domain-specific languages*. CreateSpace.